RÉPUBLIQUE DU SÉNÉGAL

Un Peuple - Un But - Une Foi

****

ÉCOLE NATIONALE DE LA STATISTIQUE ET DE L'ANALYSE ÉCONOMIQUE PIERRE NDIAYE

RAPPORT TECHNIQUE DETAILLEE : *PROGRESS*

Projet final d'algorithmique et programmation avancée avec python

Année académique: 2025-2026

PROGRESS

Espace Étudiant Cozy

Rapport complet du projet final

Équipe : Leslye Nkwa, Marème Diop, Trèsor Riradjim

Filière : ISE1 CL - ENSAE de Dakar

Cours : Algorithme et Programmation Python avancé

Professeur : M. Pape Adama MBOUP

Année : 2025 - 2026

Table des matières

1. Introduction
2. Contexte académique
3. Alignement avec le cours : les concepts de là POO
3.1 La Programmation Orientée Objet (POO)
3.2 L'encapsulation
3.3 L'héritage
3.4 Le polymorphisme
4. Outils techniques utilisés
4.1 Les langages
4.2 Le framework Django
4.3 GitHub et le versioning
4.4 GitHub Actions (CI/CD)
4.5 Render (déploiement)
4.6 UptimeRobot (monitoring)
5. Architecture complété de l'application
5.1 Vue d'ensemble
5.2 Les dossiers principaux (les apps)
5.3 Les dossiers de support
5.4 Les fichiers a là racine
5.5 Comment tout fonctionne ensemble
6. Les modèles de donnees
7. Les fonctionnalités par onglet
8. Le système de calcul des moyennes
9. Le système de predictions
10. Le calendrier et les rappels
11. La génération de documents PDF
12. Conclusion

1. Introduction

Il est 23h un dimanche soir. Aminata, étudiante en première année à l'ENSAE de Dakar, est assise devant son bureau. Demain, elle à un DS de statistiques à 8h. Mais elle ne retrouve plus ses notes du dernier contrôle. Elle ne sait plus si elle à 14 où 15 en TP. Elle ne se souvient plus du coefficient de la matière. Elle a noté la date de l'examen quelque part, mais elle ne sait plus où. Et elle se demande : "Si j'obtiens 16 demain, est-ce que j'aurai ma moyenne de 14 ?"

Ce scenario, on l'a tous vécu. On est trois étudiants à l'ENSAE — Leslye, Mareme et Trèsor — et on s'est dit : pourquoi ne pas créer un outil qui résout exactement ce problème ?

C'est comme ça qu'est ne PROGRESS : un espace étudiant "cozy", une application web qui centralise tout ce dont un étudiant à besoin. Pas une usine à gaz. Pas un logiciel complique. Juste un outil simple, agréable à utiliser, qui fait exactement ce qu'il faut :

- Suivre ses notes et calculer ses moyennes automatiquement
- Savoir exactement quelle note il faut au prochain contrôle pour atteindre son objectif
- Planifier ses événements avec des rappels qui arrivent au bon moment
- Garder le contact avec ses professeurs
- Générer un bulletin de notes en PDF
- Personnaliser son espace (thème, couleurs, police)

Le nom "PROGRESS" n'est pas un hasard. C'est exactement l'idée : suivre sa progression, avancer, s'améliorer. Et le mot "cozy" dans le sous-titre, c'est parce qu'on voulait que l'application donne envie d'être utilisée. Pas un tableau Excel froid et impersonnel, mais un vrai espace chaleureux, avec des couleurs douces, des pastels, un design inspiré de Notion.

Ce rapport présente le projet dans son intégralité : le contexte académique qui l'a motivé, les concepts de programmation que nous avons appliqués, les outils techniques utilisés, l'architecture détaillée de l'application, et ce que nous en avons appris.

2. Contexte académique

2.1 Le cadre du projet

Ce projet a été réalisé dans le cadre du cours d'Algorithme et Programmation Python avance, dispensé en première année du cyclé ISE (Ingénieur Statisticien Economiste) à l'ENSAE (Ecole nationale de la Statistique et de l'Analyse économique) de Dakar.

Après environ 20 heures de cours couvrant les bases de Python, la programmation orientée objet, les structures de données avancées et les bonnes pratiques de développement, il nous a été demande de réaliser un projet concret mettant en pratique ces apprentissages.

2.2 Le problème identifié

En tant qu'étudiants à l'ENSAE, nous avons identifié un problème récurrent : la gestion académique personnelle est éclatée entre plusieurs outils. Les notes sont sur des bouts de papier ou dans des fichiers Excel éparpillés. Les dates d'examens sont dans le téléphone, où oubliées. Les contacts des professeurs sont dispersés entre WhatsApp, les mails et les carnets. Et personne ne sait exactement quelle note il lui faut pour atteindre sa moyenne cible.

Il n'existe pas, à notre connaissance, d'outil gratuit, simple et adapte aux étudiants africains francophones qui centralise tout cela. Les solutions existantes (Pronote, Campus, etc.) sont soit payantes, soit réservées aux administrations, soit trop complexes pour un usage personnel.

2.3 La solution proposée

PROGRESS répond à ce besoin en proposant une application web accessible depuis n'importe quel navigateur (téléphone, ordinateur, tablette), gratuite, en français, et pensée pour les étudiants de l'ENSAE. L'application permet de :

- Centraliser toutes les matières, notes et coefficients en un seul endroit
- Calculer automatiquement les moyennes pondérées (par matière et générale)
- Fixer des objectifs académiques et suivre sa progression en temps réel
- Prédire ses résultats : "si j'obtiens X, ma moyenne sera Y"
- Planifier ses événements avec un calendrier interactif et des rappels
- Gérer ses contacts professeurs (email, disponibilités, notes personnelles)
- Générer des bulletins de notes en PDF
- Personnaliser son interface (thème clair/sombre, couleurs, police)

2.4 Pourquoi ce choix de projet

Nous avons choisi ce projet pour trois raisons principales :

- Il répond à un vrai besoin que nous vivons au quotidien en tant qu'étudiants
- Il couvre un large spectre de compétences techniques : POO, bases de données, interfaces web, génération de fichiers, déploiement en production
- Il est suffisamment ambitieux pour montrer ce qu'on a appris, tout en restant réalisable dans le temps imparti

3. Alignement avec le cours : les concepts de la POO

Le cours de Python avance nous à enseigne la Programmation Orientée Objet (POO). Dans cette section, nous définissons chaque concept clairement, puis nous montrons exactement où et comment il est utilisé dans PROGRESS.

3.1 La Programmation Orientée Objet (POO)

Définition

La Programmation Orientée Objet est une façon d'organiser son code en "objets". Un objet, c'est une entité qui regroupe des données (ses attributs) et des actions (ses méthodes). C'est comme un formulaire : il a des champs (nom, prénom, date) et des actions qu'on peut faire dessus (remplir, valider, envoyer).

En Python, on créée des objets à partir de "classes". Une classe, c'est le modèle, le plan de construction. L'objet, c'est la version concrète, remplie avec de vraies données.

Exemple simple : la classe "Étudiant" définit qu'un étudiant à un nom, un prénom et une classe. L'objet "Aminata Diallo, ISE1 CL" est une instance concrète de cette classe.

Application dans PROGRESS

Notre application utilisé 10 classes (modèles) qui représentent chacune une entité du monde réel :

Classe	Représente	Fichier
Student	Un étudiant inscrit	core/models.py
Course	Une matière du semestre	core/models.py
Grade	Une note individuelle	core/models.py
AcademicGoal	Un objectif de moyenne	dashboard/models.py
Event	Un événement du calendrier	calendar_app/models.py
Professor	Un professeur	contacts/models.py
ProfessorNote	Une note sur un professeur	contacts/models.py
ContactLog	Un contact avec un professeur	contacts/models.py
Bulletin	Un bulletin PDF généré	contacts/models.py
UserPréférence	Les préférences d'affichage	settings_app/models.py

En plus des modèles, nous avons deux classes de services qui regroupent la logique métier :

- Analyzer (grades/services.py) : toutes les méthodes de calcul (moyennes, prédictions, classement des difficultés)
- DocumentGenerator (contacts/services.py) : génération des bulletins et exports PDF

3.2 L'encapsulation

Définition

L'encapsulation, c'est le principe de cacher la complexité interne d'un objet et de n'exposer que ce qui est nécessaire. C'est comme une voiture : tu appuies sur l'accélérateur (interface simple), mais tu n'as pas besoin de savoir comment le moteur fonctionne en interne (complexité cachée).

En Python, l'encapsulation se fait principalement avec :

- Les propriétés (@property) : des méthodes qui se comportent comme des attributs
- Les validateurs : des règles qui s'appliquent automatiquement aux données
- Les méthodes privées : des fonctions internes précédées de _ (underscore)

Application dans PROGRESS

Exemple 1 : La propriété needs_reminder dans le modèle Event. Quand le code veut savoir si un événement doit déclencher un rappel, il écrit simplement "event.needs_reminder". Mais derrière, cette propriété fait un calcul complexe :

- Elle vérifie que le rappel est actif et que l'événement n'est pas terminé
- Elle construit le datetime de l'événement à partir de la date et de l'heure
- Elle calculé le delta du rappel (jours + heures + minutes)
- Elle compare le moment actuel avec la fenêtre de rappel

Tout ce calcul est caché. Le code qui appelle "event.needs_reminder" reçoit juste True ou False. C'est ça l'encapsulation.

Exemple 2 : La méthode get_average() dans le modèle Course. Pour obtenir la moyenne d'une matière, on écrit "course.get_average()". La formule de calcul (somme pondérée des notes divisée par la somme des poids) est cachée à l'intérieur de la méthode. Le reste du code n'a pas besoin de connaitre la formule.

Exemple 3 : Les validateurs sur le modèle Grade. Le champ 'value' à un MinValueValidator(0) et un MaxValueValidator(20). Si quelqu'un essaie d'enregistrer une note de -3 où de 25, Django refuse automatiquement. La règle de validation est encapsulée dans le modèle.

Exemple 4 : Les fonctions privées _get_student(request) dans les vues. Le underscore au début indique que c'est une fonction interne, pas destinée à être appelée de l'extérieur. Elle encapsule là logique de récupération du profil étudiant.

3.3 L'héritage

Définition

L'héritage, c'est quand une classe "hérite" des propriétés et méthodes d'une autre classe. C'est comme un formulaire pré-rempli : au lieu de tout recréer de zéro, tu pars d'un modèle existant et tu ajoutes ou modifiés ce dont tu as besoin.

La classe dont on hérite s'appelle la "classe parente" (ou classe mère). La classe qui hérite s'appelle la "classe enfant" (ou classe fille). En Python, l'héritage s'écrit : class Enfant(Parent).

Application dans PROGRESS

Tous nos modèles héritent de django.db.models.Model. C'est là classe parente fournie par Django. Grace à cet héritage, nos modèles obtiennent automatiquement :

- La capacité de se sauvegarder en base de donnees (méthode save())
- La capacité de se supprimer (méthode delete())
- Un identifiant unique auto-incremente (champ id)
- La capacité d'être recherche et filtre (queryset : objects.filter(), objects.all())
- La validation automatique des champs

Exemple concret :

```
class Student(models.Model):     # herite de Model
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    level = models.CharField(choices=LEVEL_CHOICES)

# Grace a l'heritage, on peut faire :
student = Student(first_name='Aminata', last_name='Diallo')
student.save()                    # methode heritee de Model
Student.objects.filter(level='ISE1CL')  # queryset herite
```

Sans l'héritage, il faudrait réécrire tout ce code pour chaque modèle. Avec l'héritage, Django fait le travail pour nous. Nos 10 modèles bénéficient tous de ces capacités sans que nous ayons à les coder.

3.4 Le polymorphisme

Définition

Le polymorphisme, c'est la capacité d'un même code à se comporter différemment selon le contexte. Le mot vient du grec : "poly" (plusieurs) + "morphe" (formes). C'est comme un bouton "Envoyer" : sur un formulaire email, il envoie un mail. Sur un formulaire de commande, il passe la commande. Même interface, comportement différent.

Application dans PROGRESS

Nous utilisons le polymorphisme de plusieurs façons :

1. Polymorphisme par type d'événement : Le modèle Event a un champ event_type qui peut être COURSE, EXAM, HOMEWORK, REVISION ou PERSONAL. La propriété color_classes retourne des classes CSS différentes selon le type :

```
# Meme propriete, resultat different selon le type
cours.color_classes   -> {'bg': 'bg-blue-50', 'text': 'text-blue-700'}
examen.color_classes  -> {'bg': 'bg-red-50', 'text': 'text-red-700'}
devoir.color_classes  -> {'bg': 'bg-amber-50', 'text': 'text-amber-700'}
```

2. Polymorphisme par type de document : Le DocumentGenerator généré deux types de PDF (bulletin et statistiques) avec des mises en page complètement différentes, mais en utilisant la même base de données et les mêmes modèles.

3. Polymorphisme par thème : Le même template HTML s'affiche différemment selon les préférences de l'utilisateur. En mode clair, le fond est beige clair. En mode sombre, le fond est gris fonce. Même code, rendu différent.

4. Outils techniques utilisés

Cette section expliqué chaque outil technique utilisé dans le projet, de façon simple et accessible, même pour quelqu'un qui n'a jamais programme.

4.1 Les langages

Python

Python est le langage principal du projet. C'est le langage que nous avons appris en cours. Il sert à écrire toute la logique de l'application : les calculs de moyennes, la gestion des utilisateurs, la génération de PDF, la communication avec la base de données, etc. C'est le "cerveau" de l'application.

HTML

HTML (HyperText Markup Language) est le langage qui structure les pages web. Il définit les éléments : titres, paragraphes, boutons, formulaires, tableaux, images. C'est le "squelette" de chaque page.

CSS (via Tailwind)

CSS (Cascading Style Sheets) donne le style aux pages : couleurs, tailles, espacements, animations. Nous utilisons Tailwind CSS, un framework qui permet d'écrire le style directement dans le HTML avec des classes utilitaires. Par exemple, "bg-blue-50 text-sm rounded-md" donne un fond bleu clair, du texte petit et des coins arrondis.

JavaScript (via Alpine.js)

JavaScript rend les pages interactives : ouvrir un formulaire sans recharger la page, afficher/cacher un menu, animer une notification. Nous utilisons Alpine.js, un micro-framework très léger qui s'intégré directement dans le HTML. Par exemple, le système de notification qui affiché les rappels en temps réel utilisé JavaScript pour interroger le serveur toutes les 30 secondes.

4.2 Le framework Django

Django est un framework web Python. Un framework, c'est un ensemble d'outils préconstruits qui évitent de tout refaire de zéro. C'est comme un kit de construction IKEA : les planches sont pré-découpées, les vis sont fournies, il suffit de suivre le plan et d'assembler.

Django fournit :

- Un système de routage URL (chaque adresse web correspond à une fonction Python)

- Un ORM (Object-Relational Mapper) qui permet de manipuler la base de données avec du Python au lieu du SQL
- Un moteur de templates pour générer du HTML dynamique
- Un système d'authentification complet (inscription, connexion, mots de passe)
- Une interface d'administration automatique
- Un système de migrations pour gèrer l'évolution de la base de données

Django suit le pattern MTV (Model-Template-View) :

- Model : définit la structure des données (les tables)
- Template : définit l'affichage (le HTML)
- View : définit la logique (que faire quand l'utilisateur clique)

4.3 GitHub et le versioning

GitHub est la plateforme qui héberge le code source du projet. C'est comme Google Drive, mais pour du code. Chaque modification est enregistrée avec une description (un "commit"), ce qui permet de :

- Travailler à plusieurs sans écraser le travail des autres
- Revenir en arrière si une modification casse quelque chose
- Avoir une copie de sauvegarde du code en ligne
- Montrer le code au professeur où a un recruteur

Notre repository est accessible à l'adresse : github.com/Deep5Mind/progress-app

4.4 GitHub Actions (CI/CD)

CI/CD signifie Intégration Continue / Déploiement Continu. C'est un système automatique qui vérifié le code à chaque push sur GitHub.

Concrètement, à chaque fois qu'on pousse du code, un serveur de GitHub :

- Télécharge notre code
- Installé Python et les dépendances
- Exécute "python manage.py check" pour vérifier la configuration Django
- Exécuté "python manage.py test" pour lancer les tests

Si tout passe, on voit une coche verte sur GitHub. Si quelque chose échoué, une croix rouge apparait avec le détail de l'erreur. Cela permet de détecter les problèmes immédiatement, avant qu'ils n'arrivent en production.

4.5 Render (déploiement)

Render est là plateforme qui hébergé notre application en ligne. C'est un serveur dans le cloud qui fait tourner notre code Python 24h/24. Quand quelqu'un tape l'adresse de notre app dans son navigateur, c'est Render qui reçoit la requête, exécute le code Django, et renvoie la page HTML.

Render est connecté à notre repository GitHub. A chaque push, Render re-deploie automatiquement la nouvelle version. C'est le "CD" (Déploiement Continu) de notre pipeline CI/CD.

Notre application est accessible à l'adresse : progress-app-ftcy.onrender.com

4.6 UptimeRobot (monitoring)

Sur le plan gratuit de Render, l'application se met en veille après 15 minutes d'inactivité. La première visite après une période d'inactivité prendrait 30 à 50 secondes (le temps de "réveiller" le serveur).

Pour éviter cela, nous utilisons UptimeRobot, un service gratuit qui "visite" notre site toutes les 5 minutes. Le serveur croit qu'il y a toujours du trafic et ne se met jamais en veille. Résultat : notre application répond instantanément a tout moment.

5. Architecture compléte de l'application

5.1 Vue d'ensemble

Le projet est organisé en dossiers, chacun ayant un rôle précis. C'est comme un immeuble : chaque étage à une fonction (commerce au rez-de-chaussée, bureaux au premier, appartements au-dessus). Dans Django, chaque dossier principal s'appelle une "app" et gère un domaine fonctionnel.

Voici là structure complété du projet :

```
cozy_student_app/
├── config/          # Configuration Django (settings, urls)
├── core/            # Modeles centraux (Student, Course, Grade)
├── dashboard/       # Onglet 1 : tableau de bord et objectifs
├── grades/          # Onglet 2 : notes, analyses, predictions
├── calendar_app/    # Onglet 3 : calendrier et rappels
├── contacts/        # Onglet 4 : repertoire professeurs, PDF
├── accounts/        # Authentification (login, inscription)
├── settings_app/    # Parametres utilisateur
├── templates/       # Templates HTML partages
├── static/          # Fichiers statiques (images, CSS)
├── manage.py        # Point d'entree Django
├── requirements.txt # Dependances Python
├── render.yaml      # Configuration deploiement Render
├── build.sh         # Script de build production
└── Procfile         # Commande de demarrage production
```

5.2 Les dossiers principaux (les apps)

config/ — Le centre de contrôle

C'est le dossier de configuration de Django. Il contient settings.py (tous les paramètres de l'application : base de données, apps installées, middleware, fuseau horaire, etc.), urls.py (la table

de routage principale qui redirigé chaque URL vers la bonne app), et wsgi.py (le point d'entrée pour le serveur de production).

core/ — Le coeur des donnees

Contient les trois modèles fondamentaux : Student (l'étudiant), Course (la matière) et Grade (la note). Ces modèles sont utilisés par toutes les autres apps. C'est le socle de l'application. Core n'a pas de vues ni de templates propres : il fournit juste les données que les autres apps affichent.

dashboard/ — Le tableau de bord (Onglet 1)

Affiché la vue d'ensemble : moyenne générale, nombre de matières et de notes, barres de progression par matière, objectifs académiques, événements de la semaine, rappels urgents et citation motivante. C'est la première page que l'étudiant voit après connexion.

grades/ — Les notes et analyses (Onglet 2)

Gère l'ajout, la modification et la suppression des matières et des notes. Affiche les moyennes pondérées, les graphiques d'évolution (Chart.js), le classement des matières en difficulté, et le système de prédictions. Contient aussi la classe Analyzer (services.py) qui fait tous les calculs.

calendar_app/ — Le calendrier (Onglet 3)

Affiche un calendrier mensuel interactif avec une grille de jours. Permet de créer des événements de 5 types (cours, examen, devoir, révision, personnel) avec des rappels configurables en jours, heures et minutes. Inclut aussi une API JSON pour les notifications en temps réel.

contacts/ — Les contacts et documents (Onglet 4)

Répertoire de professeurs avec leurs informations (email, statut, disponibilités). Permet d'ajouter des notes personnelles et un historique de contacts. Contient aussi le système de génération de bulletins PDF (DocumentGenerator) et l'historique des documents générés.

accounts/ — L'authentification

Gère l'inscription, la connexion et la déconnexion des utilisateurs. Utilisé le système d'authentification intégré de Django. A l'inscription, créé automatiquement un profil Student lie au compte.

settings_app/ — Les parametrès

Permet à l'utilisateur de personnaliser son interface : thème (clair/sombre), police (Inter, Poppins, Nunito, Lora, JetBrains Mono), couleur d'accent (vert sauge, bleu océan, lavande, pèche, rose). Permet aussi de modifier son profil, changer son mot de passe, exporter ses données en JSON et réinitialiser ses préférences.

5.3 Les dossiers de support

templates/

Contient les templates HTML partages entre toutes les apps. Le fichier base.html est le template maitre : il définit la structure commune (sidebar, topbar, zone de contenu, notifications). Chaque page de l'application "hérite" de ce template et remplit uniquement la zone de contenu. Le sous-dossier components/ contient les composants réutilisables : la barre latérale (_sidebar.html), là barre du haut (_topbar.html), les notifications toast (_toast.html), etc.

static/

Contient les fichiers statiques qui ne changent pas : images (le logo), et eventuels fichiers CSS où JavaScript personnalisés. En production, WhiteNoise sert ces fichiers de façon optimisee.

migrations/ (dans chaque app)

Chaque app qui a des modèles contient un sous-dossier migrations/. Ces fichiers sont des "plans de construction" pour la base de données. Quand on ajoute un champ à un modèle, Django créé un fichier de migration qui décrit le changement. Ensuite, "migrate" applique ce changement a là base. Ces fichiers sont versionnés sur GitHub pour que la base de production puisse être construite de la même façon.

5.4 Les fichiers a là racine

manage.py

Point d'entree de Django. C'est le fichier qu'on appelle pour toutes les commandes : runserver, migrate, createsuperuser, check, etc.

requirements.txt

Liste de tous les packages Python nécessaires. Quand quelqu'un clone le projet, il fait 'pip install -r requirements.txt' pour installer tout d'un coup.

.gitignore

Liste des fichiers que Git doit ignorer (secrets, base de données, environnement virtuel). Empêche les fichiers sensibles d'aller sur GitHub.

.env

Fichier de variables d'environnement locales (secrets). N'est JAMAIS envoyé sur GitHub.

render.yaml

Configuration de déploiement pour Render. Décrit le service web et la base de données à créer.

build.sh

Script exécuté par Render à chaque déploiement. Installé les packages, collecte les fichiers statiques, applique les migrations.

Procfile

Dit à Render quelle commande utiliser pour démarrer le serveur (gunicorn au lieu de runserver).

README.md

Documentation du projet affichée sur la page GitHub. Expliqué comment installer et lancer le projet.

5.5 Comment tout fonctionne ensemble

Voici ce qui se passe, étape par étape, quand un étudiant utilise PROGRESS :

Scenario : Aminata ouvre son navigateur et va sur progress-app-ftcy.onrender.com/grades/

- 1. Le navigateur envoie une requête HTTP GET à l'adresse /grades/
- 2. Le serveur Gunicorn (sur Render) reçoit la requête et la transmet à Django
- 3. Django consulte config/urls.py et voit que /grades/ correspond à l'app 'grades'
- 4. Django consulte grades/urls.py et voit que la racine / correspond a là vue index
- 5. Django exécuté la fonction index() dans grades/views.py
- 6. La vue vérifié qu'Aminata est connectée (@login_required)
- 7. La vue récupère le profil Student d'Aminata depuis la base de données
- 8. La vue récupère toutes ses matières (Course) avec leurs notes (Grade)
- 9. La vue appelle Analyzer pour calculer les moyennes, prédictions, classements
- 10. La vue transmet toutes ces données au template grades/index.html
- 11. Django généré le HTML final en combinant le template avec les données
- 12. Le HTML est envoyé au navigateur d'Aminata
- 13. Le navigateur affiché la page avec les styles Tailwind et les interactions Alpine.js

Tout ce processus se fait en quelques millisecondes. L'utilisateur ne voit qu'une page qui s'affiche. Mais derrière, 13 étapes se sont enchaines.

6. Les modèles de données

Les modèles définissent la structure de la base de données. Chaque modèle correspond à une table, et chaque champ correspond à une colonne. Voici les principaux modèles et leurs relations :

Schéma des relations :

```
User (Django)
  |-- (1:1) --> Student
  |               |-- (1:N) --> Course
  |               |               |-- (1:N) --> Grade
  |               |-- (1:N) --> AcademicGoal
  |               |-- (1:N) --> Event
  |               |-- (1:N) --> Professor
  |               |               |-- (1:N) --> ProfessorNote
  |               |               |-- (1:N) --> ContactLog
  |               |-- (1:N) --> Bulletin
  |-- (1:1) --> UserPreference
```

(1:1) signifie "un pour un" : un User a exactement un Student. (1:N) signifie "un pour plusieurs" : un Student a plusieurs Courses, chaque Course a plusieurs Grades, etc.

Modèle	Champs principaux	Relations
Student	first_name, last_name, level	1 User, N Courses, N Events
Course	name, coefficient	1 Student, N Grades
Grade	value (/20), grade_type, weight (%)	1 Course
AcademicGoal	target_average, semester	1 Student, 1 Course (optionnel)
Event	title, event_type, date, time, reminder_days/hours/minutes	1 Student
Professor	name, email, status, other_job, subjects	1 Student, N Notes, N Logs
UserPréférence	theme, font, accent_color, motivational_quote	1 User

7. Les fonctionnalités par onglet

Onglet 1 : Tableau de bord

- Message de bienvenue personnalisé avec le prénom de l'étudiant
- Citation motivante éditable (clic pour modifier)
- 4 cartes statistiques : moyenne générale, objectif, nombre de matières, nombre de notes
- Barres de progression par matière avec code couleur (vert/bleu/jaune/rose)
- Gestion des objectifs académiques (général et par matière)
- Résumé des événements de la semaine
- Rappels urgents (événements proches avec rappel active)

Onglet 2 :Notes & Analyses

- Ajout, modification et suppression de matières avec coefficient
- Ajout, modification et suppression de notes avec type, poids (%) et date
- Validation des poids : le total par matière ne peut pas dépasser 100%
- Calcul automatique des moyennes pondérées (par matière et générale)
- Graphique d'évolution de la moyenne au fil du temps (Chart.js)
- Graphique comparatif de toutes les matières
- Prédictions : "si j'obtiens X, ma moyenne en cette matière sera Y et ma moyenne générale sera Z"
- Note requise pour atteindre un objectif
- Classement des matières par difficulté

Onglet 3 : Calendrier

- Vue mensuelle interactive avec grille de jours
- 5 types d'événements avec couleurs distinctes (cours, examen, devoir, revision, personnel)
- Ajout, modification et suppression d'événements depuis la grille, la sidebar et la section du jour
- Rappels configurables en jours, heures et minutes

- Notifications en temps réel (bannière + notification navigateur)
- Navigation entre les mois
- Marquage des événements comme terminés

Onglet 4 :Contacts & Documents

- Répertoire des professeurs en cartes horizontales scrollables
- Informations : nom, email, téléphone, statut (permanent/vacataire), autre métier, disponibilités
- Notes personnelles sur chaque professeur
- Historique de contacts (email, rendez-vous, permanence)
- Génération de bulletins PDF avec appréciations automatiques
- Export statistiques en PDF avec analyse des difficultés

Paramètres

- Thème : mode clair où mode sombre
- Police : 5 choix (Inter, Poppins, Nunito, Lora, JetBrains Mono)
- Couleur d'accent : 5 choix (vert sauge, bleu océan, lavande, pêche, rose)
- Modification du profil (prénom, nom, email, classe)
- Changement de mot de pass
- Export des données en JSON
- Réinitialisation des préférences et suppression des données

8. Le système de calcul des moyennes

8.1 Moyenne d'une matière

La moyenne d'une matière est une moyenne pondérée par les poids des notes. Chaque note à un poids en pourcentage (1-100%) qui détermine son importance.

```
Moyenne = somme(note_i x poids_i) / somme(poids_i)
```

Exemple : en Statistiques, trois notes :
- DS : 12/20 (poids 30%)
- TP : 15/20 (poids 20%)
- Examen : 14/20 (poids 50%)

Moyenne = (12x30 + 15x20 + 14x50) / (30+20+50) = 1360 / 100 = 13.60/20

8.2 Moyenne générale

La moyenne générale est une moyenne pondérée par les coefficients des matières.

```
Moy. Gen. = somme(moyenne_matiere_i x coefficient_i) / somme(coefficient_i)
```

Exemple :
- Statistiques : 13.60 (coeff 3)
- Python : 16.00 (coeff 2)
- Economie : 11.70 (coeff 2)

Moy. Gen. = (13.60x3 + 16.00x2 + 11.70x2) / (3+2+2) = 96.60 / 7 = 13.80/20

8.3 La différence entre poids et coefficient

Le POIDS s'applique à une note individuelle DANS une matière. Le COEFFICIENT s'applique à une matière entière DANS la moyenne générale. Ce sont deux niveaux de pondération différents.

	POIDS	COEFFICIENT
S'applique a	Une note	Une matière
Sert à calculér	Moyenne de là matière	Moyenne générale
Valeur	1 à 100 (%, total = 100)	Décimal (ex: 1.0, 3.0)
Exemple	DS 30% + TP 20% + Exam 50%	Maths coeff 3, Sport coeff 1

9. Le système de predictions

9.1 Prediction de moyenne

L'étudiant peut simuler : "Si j'obtiens X au prochain contrôle (avec un poids de Y%), quelle sera ma moyenne dans cette matière et quelle sera ma moyenne générale ?"

```
Nouvelle moy. matiere = (somme_actuelle + note_hypo x poids_hypo)
                      / (total_poids + poids_hypo)

Nouvelle moy. generale = somme(moy_matiere_i x coeff_i) / somme(coeff_i)
   (en remplacant la moyenne de la matiere par la nouvelle)
```

9.2 Note requise pour un objectif

Inversement : "Quelle note minimale me faut-il pour atteindre 14/20 ?"

```
Note requise = (objectif x (total_poids + poids_prochain) - somme_actuelle)
             / poids_prochain
```

Si le résultat dépassé 20, l'objectif est mathématiquement impossible en un seul contrôle. L'application l'indique clairement.

10. Le calendrier et les rappels

Le calendrier affiché une grille mensuelle de 6 semaines x 7 jours. Chaque événement est affiché avec sa couleur de type dans là case du jour.

10.1 Les rappels

Les rappels sont configurables avec une précision en jours, heures et minutes. Le système de notification fonctionne en temps réel grâce à un mécanisme de polling : un script JavaScript dans le navigateur interroge le serveur toutes les 30 secondes via une API JSON (/calendar/api/reminders/). Si un rappel est actif, une bannière apparait en haut de là page et une notification navigateur est envoyée.

Le rappel est actif quand : datetime_événement - délai_rappel <= maintenant <= datetime_événement + 15 min

11. La génération de documents PDF

L'application généré deux types de documents PDF avec la bibliothèque ReportLab :

Bulletin de notes

Document officiel avec : informations de l'étudiant, tableau de toutes les matières avec notes, moyennes et appréciations automatiques (Très bien >= 16, Bien >= 14, Assez bien >= 12, Passable >= 10, Insuffisant < 10). Un filigrane "DOCUMENT PROVISOIRE" est ajouté en diagonale.

Export statistiques

Document détaillé avec : résumé général, tableau comparatif des matières (moyenne, objectif, écart, statut), et analyse des matières en difficulté.

12. Conclusion

12.1 Ce que nous avons appris

Ce projet nous a permis d'aller bien au-delà du cours. Nous avons appris à :

- Concevoir une application compléte de A à Z, du modèle de données au déploiement
- Appliquer la POO dans un contexte réel : nos classes ne sont pas des exercices, elles résolvent de vrais problèmes
- Utiliser un framework professionnel (Django) et comprendre son architecture MTV
- Gèrer une base de données relationnelle avec des relations complexes (OneToOne, ForeignKey, cascades)
- Créer une interface utilisateur moderne et responsive
- Déployer une application en production sur un serveur distant
- Mettre en place un pipeline CI/CD pour la qualité du code
- Générer des documents PDF professionnels par programmation
- Travailler en équipe avec Git et GitHub

12.2 Compétences développées

Domaine	Compétences
Python	POO, classes, propriétés, decorateurs, services
Web	HTTP, HTML, CSS, JavaScript, API JSON
Django	MTV, ORM, migrations, auth, middleware, context processors
Base de donnees	Modélisation relationnelle, SQL, PostgreSQL
DevOps	Git, GitHub, CI/CD, déploiement cloud, monitoring
Design	UI/UX, responsive design, accessibilité, themes
PDF	Génération programmatique avec ReportLab

12.3 Perspectives

PROGRESS pourrait évoluer dans plusieurs directions :

- Application mobile native (React Native ou Flutter)
- Système de partage entre étudiants (notes collaboratives)

- Intégration avec les systèmes académiques de l'ENSAE
- Intelligence artificielle pour des recommandations de révisions personnalisées
- Notifications push sur telephone

Ce projet est notre première application complète. Il nous a appris que la programmation, ce n'est pas juste écrire du code. C'est comprendre un problème, concevoir une solution, là construire, là tester, là déployer et la rendre accessible aux autres. C'est ce que nous avons voulu faire avec PROGRESS.

Leslye Nkwa, Marème Diop, Trésor Riradjim

ISE1 CL , ENSAE de Dakar , 2025-2026